

CBS-Bootstrapped MAPPO: Scalable Multi-Agent Path Finding Under Dynamic Obstacles and Natural Language Control

Kyaw Linn Khant
Independent Researcher
Singapore
klkhant707@gmail.com

Abstract

Multi-Agent Path Finding (MAPF) is a critical challenge in warehouse automation and multi-robot systems. Classical solvers such as Conflict-Based Search (CBS) provide optimal guarantees but are exponential in the number of agents and fundamentally cannot handle dynamic environments. Fully learned approaches suffer from catastrophic cold-start: sparse multi-agent rewards leave the policy gradient uninformative for the first hundreds of thousands of training steps, producing degenerate always-wait policies. We present **CBS-Bootstrapped MAPPO**: a three-phase curriculum that uses CBS as a *decaying reward signal*—not imitation learning—to bootstrap a transformer-based multi-agent reinforcement learning policy. The CBS bonus anneals from weight 1.0 to 0.0 over 500k steps; by Phase C the policy is fully independent, continues to improve, and generalises beyond the teacher’s reach. We evaluate on procedurally generated warehouse maps across four difficulty levels (7×7 to 20×20 , 2–12 agents) under lifelong goal assignment. On static maps, our policy surpasses CBS at all difficulty levels (71%–95% vs. CBS 0%–61%). Crucially, under dynamic obstacles—where CBS plan invalidation rates reach 50%–100%—our policy maintains 79%–95% per-agent success rate while CBS degrades to 0%–7%. We further extend the system with a natural language zone-assignment interface: a local LLM (Qwen2.5-3B via Ollama) parses operator commands into warehouse zone targets which the pretrained MARL policy executes collision-free with no retraining. Training runs entirely on commodity Apple Silicon hardware (14 hours for 7M steps). Code and pretrained models are available at <https://github.com/KyawLinnKhant/mapf-cbs>.

1 Introduction

Multi-Agent Path Finding (MAPF) is the problem of routing N agents through a shared grid environment from their respective starts to goals, subject to the constraint that no two agents occupy the same cell (vertex conflict) or swap positions (edge conflict) at any timestep [7]. MAPF

is the core computational problem underpinning modern warehouse automation [3], multi-robot coordination, and air traffic management. Industrial deployments at scale—warehouses with dozens of robots, hospitals with autonomous delivery fleets, distribution centres running 24/7—introduce two complications that classical solvers cannot address.

Dynamic obstacles. Human workers, forklifts, and other unmodelled entities move through the environment at runtime. A plan that was valid when computed may be blocked within seconds. CBS plans on a static snapshot of the world; when an obstacle moves into a planned path, the plan is invalidated and must be recomputed from scratch. Our experiments show CBS plan invalidation rates of 51%–100% with just four moving obstacles, and full CBS failure on expert-level maps (CBS solve time exceeds the allowed budget even on static maps).

Lifelong operation. In real deployments, agents are continuously assigned new goals rather than solving a single batch [3]. The relevant metric is sustained *throughput*—goals reached per unit time—not one-shot plan optimality. Classical solvers optimise for SoC on a single batch; they have no native mechanism for lifelong operation without constant replanning.

Classical MAPF solvers. CBS [7] finds optimal solutions via a two-level conflict tree search but scales exponentially in the number of conflicts. MAPF-LNS2 [1] improves scalability via large neighbourhood search but retains the static-world assumption. In our experiments, CBS degrades to 0% success on expert-level maps (12 agents, 20×20 grid) when capped at 10,000 constraint tree nodes—a generous budget for real-time operation.

Learned MAPF approaches. PRIMAL [5] and MAPPO [10] offer $O(1)$ -per-agent inference and naturally handle environmental change, but suffer from catastrophic cold-start. In standard MAPF reward structures (+1 on goal, -0.5 on collision, -0.01 per step), random agent initialisation leads to immediate collisions and no positive reward signal. The policy gradient is dominated by collision penalties with no gradient toward coordination for the first hundreds of thousands of steps.

Our approach. We bridge these paradigms by using CBS as a *temporary reward shaper*. During Phase A (0–100k steps), CBS computes optimal actions at each step; agents receive a +0.3 bonus when their sampled action matches CBS. During Phase B (100k–500k steps), this bonus anneals linearly to zero. By Phase C (500k+ steps), the policy is entirely RL-driven. This provides the warm start needed to exit the zero-gradient region while ensuring the policy is not permanently anchored to CBS solutions—a requirement for Phase C generalisation to dynamic environments.

Key distinction. SCRIMP [9] uses CBS-generated trajectories for imitation learning, maintaining a demonstration buffer from which the policy clones behaviour. Our CBS bonus is computed at each step from the *current* policy state, not from a fixed buffer. More importantly, it decays to zero, allowing the policy to discover coordination strategies (dynamic obstacle avoidance, flow corridors, adaptive priority passing) that CBS never produces. No demonstration buffer is maintained.

Contributions. We make three contributions:

1. **CBS as annealing reward shaper:** We show that CBS used as a decaying action-match bonus—rather than imitation learning—provides stable curriculum bootstrapping for MARL on MAPF. The policy surpasses CBS performance at every difficulty level within 500k steps and continues improving for 7M total steps.
2. **Curriculum trains beyond the teacher:** On environments with dynamic obstacles where CBS plan invalidation reaches 51%–100%, our policy maintains 79%–95% per-agent success rate, while CBS+dynamic degrades to 0%–14%. The policy was not explicitly trained for dynamic obstacles; generalisation emerges from the local-reactive observation structure combined with learned coordination.
3. **LLM zone assignment:** We demonstrate a natural language interface where operator commands are parsed by a local LLM (Qwen2.5-3B, fully offline) into warehouse zone assignments, executed collision-free by the pretrained policy without retraining. The LLM handles only *intent parsing*—not path planning—cleanly separating language understanding from execution.

2 Related Work

2.1 Classical MAPF

CBS [7] solves MAPF optimally via a two-level search: a high-level constraint tree resolves agent conflicts, and a low-level A^* planner finds single-agent paths subject to constraints. CBS is complete and optimal but scales exponentially in the number of conflicts—for dense environments with many agents, the constraint tree becomes intractably large. MAPF-LNS2 [1] applies large neighbourhood search to improve scalability at the cost of optimality guarantees. LaCAM2 and EECBS offer sub-optimal but

faster solutions. All classical solvers assume a static environment and require expensive replanning when dynamic obstacles invalidate a plan.

2.2 Learning-Based MAPF

PRIMAL [5] uses ODrM* demonstrations for imitation followed by RL fine-tuning, addressing cold-start through expert demonstrations. SCRIMP (2023) [9] combines CBS-generated demonstrations with PPO in a curriculum framework, using a fixed demonstration buffer. DHC and SACHA use communication networks but do not address dynamic obstacles. MAPPO [10] demonstrated the effectiveness of PPO with centralised training for cooperative multi-agent games, but did not apply it to the MAPF cold-start problem or dynamic environments.

Our approach differs from PRIMAL and SCRIMP in a key respect: we use CBS as a *live reward signal that decays to zero*, not a fixed demonstration dataset. The policy never clones CBS trajectories; it receives a diminishing bonus for agreeing with CBS and is free to diverge as the bonus approaches zero. This allows Phase C to discover coordination strategies beyond CBS’s capability.

2.3 Reward Shaping for MARL

Potential-based reward shaping [4] guarantees policy invariance: any potential function $F(s, s') = \gamma\Phi(s') - \Phi(s)$ produces the same optimal policy as the unshaped MDP. Our CBS bonus is not potential-based—it is action-matched, not state-based—and intentionally decays to zero, breaking the invariance guarantee by design. This is *required* for Phase C: a policy that permanently receives CBS bonuses has no incentive to learn behaviour CBS cannot produce. The decay schedule ensures that by Phase C, the RL objective fully governs learning.

2.4 LLMs for Robotics and Multi-Robot Control

Recent work uses LLMs as planners for single-robot manipulation (SayCan, PaLM-E, RT-2) and as high-level task decomposers (SayPlan, LLM-MCTS). Attempts to use LLMs as direct MAPF solvers (e.g., path generation for multiple agents) consistently fail for 3+ agents: the combinatorial action space (5^N joint actions for N agents per step) exceeds LLM in-context reasoning capacity.

We use the LLM exclusively for intent parsing—extracting spatial zone assignments from natural language—and delegate all path planning and navigation to the pretrained MARL policy. This clean separation means the LLM’s failure modes (hallucinated zone names, malformed JSON) can be handled by a deterministic regex fallback without affecting the navigation policy. Ashish Kapoor’s “ChatGPT for Robotics” [2] establishes the principle that LLMs are better at generating code/intents than

direct control; our zone assignment interface applies this principle to multi-robot coordination.

2.5 Entropy Collapse in Multi-Agent RL

Policy entropy collapse—where the policy distribution concentrates on a single degenerate action—is a known failure mode in policy gradient methods. In multi-agent settings, the problem is amplified: the optimal response for agent i often degenerates to “wait for all other agents,” which minimises collision penalties but also prevents any goal completion. We observe this failure mode empirically at step 5.25M (entropy drops to 0.086, goals drop from 27 to 15) and recover by raising c_{ent} from 0.005 to 0.02. We document this as a reproducible training event with recovery mechanism.

3 Problem Formulation

3.1 MAPF and Lifelong MAPF

A MAPF instance is a tuple (G, A, s, t) where $G = (V, E)$ is an undirected grid graph, $A = \{a_1, \dots, a_N\}$ is a set of N agents, $s : A \rightarrow V$ maps agents to start positions, and $t : A \rightarrow V$ maps agents to goal positions. At each discrete timestep τ , each agent takes an action $u_i \in \{\text{STAY}, \text{N}, \text{S}, \text{E}, \text{W}\}$. A solution is *valid* if no vertex conflicts (two agents occupy the same cell) or edge conflicts (agents swap positions) occur at any timestep.

Cost metrics. The classical objective minimises *sum-of-costs* $\text{SoC} = \sum_i |\pi_i|$, where $|\pi_i|$ is the length of agent i ’s path. For lifelong MAPF, we use: (1) *goals reached per episode*—total goal completions across all agents in an episode of fixed length $T = 512$ steps, and (2) *per-agent success rate*—the fraction of agents that reach at least one goal within the episode.

Lifelong operation. Following [3], each agent is immediately assigned a new random goal upon reaching its current goal. The environment thus operates continuously without terminal states (except episode length T). This models real warehouse deployments where robots are never idle.

3.2 Dynamic Obstacles

We extend the MAPF environment with K uncontrolled obstacles. At initialisation, K obstacles are placed on free cells (not overlapping agents, walls, or goals). Each step, each obstacle moves according to a stochastic pattern sampled uniformly from: (1) *random walk*—a uniformly random valid adjacent cell, and (2) *patrol*—alternating between two fixed waypoints at distance 4–6. Obstacles are observable: the agent’s 7×7 local observation includes a dedicated channel with obstacle positions (value 0.5, distinct from other agents at value 1.0 and walls).

Crucially, dynamic obstacles are *not modelled by CBS*. CBS plans on the static map snapshot at episode start.

Any obstacle that moves into a planned path step constitutes a *plan invalidation event*. We quantify invalidation rate as the fraction of planned steps whose target cells are occupied by a dynamic obstacle when the agent attempts to move there. This metric directly measures CBS’s ability to handle dynamic environments.

3.3 Observation and Action Space

Each agent i observes a 7×7 window centred on its current position, encoded as 7 binary channels:

1. Walls (static obstacles)
 2. Other agent positions (value 1.0)
 3. Own position (value 1.0, single cell)
 4. Goal direction (4-channel one-hot: N/S/E/W toward goal)
 5. Dynamic obstacle positions (value 0.5)
 6. Goal cell position (value 1.0)
 7. Distance-to-goal encoding (scalar, normalised to $[0, 1]$)
- This $7 \times 7 \times 7 = 343$ -channel tensor is flattened to a 150-dimensional vector (after a spatial pooling layer in the MLP encoder).

The action space is $\mathcal{A} = \{0, 1, 2, 3, 4\}$ corresponding to $\{\text{stay}, \text{north}, \text{south}, \text{east}, \text{west}\}$. Actions that would move an agent into a wall or out of bounds are treated as stay (masked after sampling).

4 Method

4.1 MAPPO with Transformer Communication

Figure 1 shows the full architecture. Each agent i processes its 150D observation through a shared two-layer MLP encoder (hidden dimension 128, ReLU activations) to produce an embedding $\mathbf{e}_i \in \mathbb{R}^{128}$.

Transformer communication. All N embeddings $\{\mathbf{e}_i\}_{i=1}^N$ are processed by a two-layer multi-head self-attention transformer [8] (4 heads, 128-dimensional, pre-norm LayerNorm, dropout 0.0 during evaluation). The multi-head attention is:

$$\text{MHA}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

where $M_{ij} = -\infty$ if agent j is masked (inactive), 0 otherwise. This padding mask enables variable- N operation: a checkpoint trained with $N = 12$ can be evaluated with $N = 8$ by masking positions 8–11. This is critical for real warehouse deployments where some robots may be offline, charging, or newly added.

Centralised Training, Decentralised Execution (CTDE). The context-enriched embedding $\tilde{\mathbf{e}}_i$ from the transformer feeds two heads:

MAPPO Architecture with Transformer Communication

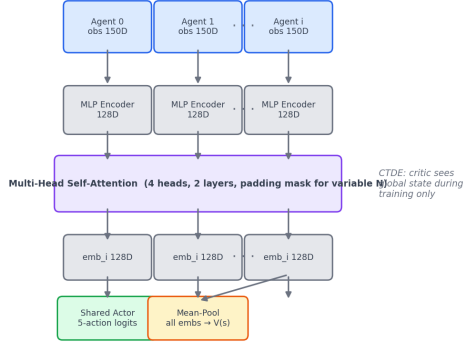


Figure 1: MAPPO architecture with transformer communication. The MLP encoder is shared across all agents. The transformer uses a padding mask to support variable numbers of active agents (padding mask = 0 for inactive agent slots). The centralised critic mean-pools all agent embeddings during training only; at deployment, only the actor is used.

- **Shared actor:** $\pi_\theta(a_i|\tilde{\mathbf{e}}_i)$ —a linear layer producing logits over 5 actions. Each agent uses its own $\tilde{\mathbf{e}}_i$ only (decentralised execution).
- **Centralised critic:** $V_\phi(s) = f_\phi(\frac{1}{N} \sum_i \tilde{\mathbf{e}}_i)$ —a linear layer applied to the mean-pooled embedding over all agents. The critic thus sees the full joint state implicitly through the mean-pooled representation.

Training objective. Standard PPO clipped surrogate with entropy regularisation:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{CLIP}} - c_{\text{ent}} \mathcal{H}[\pi_\theta] + c_v \mathcal{L}_V \quad (1)$$

with clipping $\epsilon = 0.2$, $c_{\text{ent}} = 0.02$ (raised from 0.005 at step 5.25M to recover from entropy collapse), $c_v = 0.5$. Per-agent advantages are computed via GAE [6] ($\lambda = 0.95$, $\gamma = 0.99$) and normalised across agents per rollout.

4.2 CBS-Bootstrapped Three-Phase Curriculum

Figure 2 illustrates the CBS bonus annealing schedule and difficulty curriculum progression.

Phase A (0–100k steps): CBS oracle warm start. At each step τ , the CBS oracle computes the optimal joint action a_i^* for all agents given the current grid state (walls, agent positions, goals). If agent i ’s sampled action a_i matches a_i^* , it receives an additional shaped reward:

$$r_i^{\text{CBS}} = \delta_0 \cdot \mathbf{1}[a_i = a_i^*], \quad \delta_0 = 0.3$$

The CBS computation uses a node-capped variant ($\text{max_ct_nodes} = 10,000$) to bound wall-clock cost per step. When the node budget is exceeded, CBS returns no recommendation and no bonus is applied.

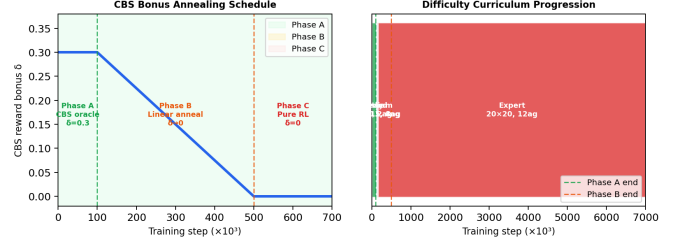


Figure 2: *Left:* CBS bonus δ annealing over training steps. Phase A maintains $\delta = 0.3$; Phase B anneals linearly to zero over 400k steps; Phase C has zero CBS dependency. *Right:* Difficulty curriculum progression. Rapid advancement (easy→expert by step 164k) is driven by the CBS warm start; all transitions occur within Phase A/B.

Table 1: Difficulty levels and map configurations.

Level	Grid	Agents	Obstacles	Density
Easy	7×7	2	0	4.1%
Medium	11×11	4	0	3.3%
Hard	15×15	8	0	3.6%
Expert	20×20	12	0	3.0%

Phase B (100k–500k steps): linear anneal. δ decays linearly from $\delta_0 = 0.3$ to 0.0:

$$\delta(\tau) = \delta_0 \cdot \left(1 - \frac{\tau - \tau_{\text{wup}}}{\tau_{\text{ann}} - \tau_{\text{wup}}}\right)$$

where $\tau_{\text{wup}} = 100,000$ and $\tau_{\text{ann}} = 500,000$. The RL gradient progressively dominates as $\delta \rightarrow 0$.

Phase C (500k+ steps): pure RL. $\delta = 0$. No CBS dependency. The policy discovers coordination strategies unavailable to CBS, including dynamic obstacle avoidance and flow corridors.

Difficulty curriculum. Maps are generated procedurally at four levels (Table 1). The curriculum auto-advances when per-agent success rate exceeds 70% and auto-regresses when it falls below 40% over a rolling 50-episode window. We observe rapid transitions: easy→medium at step 121k, medium→hard at 136k, hard→expert at 164k—all within Phase B. This rapid progression is a direct effect of the CBS bootstrap: at Phase A, CBS guides agents to near-optimal actions, driving quick success-rate increases.

Base reward. $r_i = +1.0$ on goal reached, -0.5 on collision (agent–agent or agent–obstacle), -0.01 per timestep (step penalty). No additional shaping beyond the CBS bonus.

4.3 Dynamic Obstacle Training

During dynamic evaluation, dynamic obstacles appear in the observation but are absent from the CBS planning oracle. The oracle plans on the static map; agents that face a dynamic obstacle during execution must react without CBS guidance. This creates the “teacher fails” scenario:

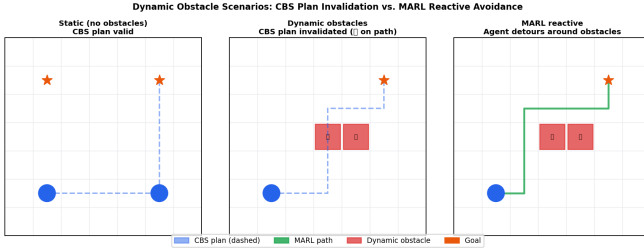


Figure 3: Dynamic obstacle scenarios. *Left*: Static environment—CBS plan (dashed blue) is valid. *Centre*: Dynamic obstacles (red) block the CBS planned path—plan invalidated. *Right*: MARL agent (green path) detours reactively around obstacles while maintaining collision avoidance.

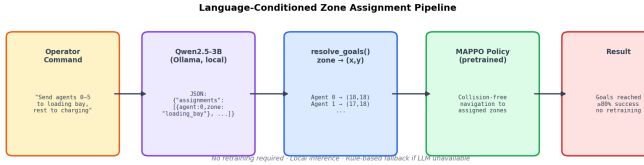


Figure 4: Language-conditioned zone assignment pipeline. The LLM handles only intent parsing (natural language $\rightarrow$ JSON zone assignments). The MARL policy handles all path planning and collision avoidance. No retraining is required for new commands or zone vocabularies.

- CBS plans a globally optimal path and executes it open-loop.
- Any dynamic obstacle entering a planned cell invalidates that plan step.
- The policy, observing its 7×7 window, detects the obstacle in the dedicated channel and can take a detour action—with no CBS bonus to guide it.

The policy generalises to dynamic obstacles without explicit dynamic-obstacle training because its 7×7 window provides real-time local sensing that CBS lacks.

Figure 3 illustrates the three scenarios: static (CBS valid), dynamic with CBS plan blocked, and MARL reactive avoidance.

4.4 Language-Conditioned Zone Assignment

Figure 4 shows the full language control pipeline.

The warehouse grid at each difficulty level has named zones (e.g., *loading_bay*, *storage_a*, *charging*, *inspection*). A local LLM receives the operator’s natural language command along with a system prompt specifying the zone vocabulary, number of agents, and output format:

SYSTEM: You are a warehouse fleet coordinator.
Parse the command. Return ONLY valid JSON:
{*"assignments"*: [{*"agent"*: <int>,

Table 2: Training hyperparameters.

Hyperparameter	Value
<i>Architecture</i>	
Hidden dimension	128
Transformer heads	4
Transformer layers	2
MLP encoder layers	2
Activation	ReLU
<i>PPO training</i>	
Learning rate	3×10^{-4} (Adam)
Clip ratio ϵ	0.2
Entropy coeff c_{ent}	$0.005 \rightarrow 0.02$ (step 5.25M)
Value coeff c_v	0.5
GAE λ	0.95
Discount γ	0.99
Rollout length	256 steps
Mini-batch size	1024
Epochs per rollout	4
<i>CBS curriculum</i>	
Phase A end	100k steps
Phase B end	500k steps
Initial bonus δ_0	0.3
Max CT nodes	10,000
<i>Curriculum scheduler</i>	
Advance threshold	70% success (50-ep window)
Regress threshold	40% success (50-ep window)
<i>Environment</i>	
Episode length T	512 steps
Dynamic obstacles K	4 (eval only)
Goal reward	+1.0
Collision penalty	-0.5
Step penalty	-0.01
<i>Hardware</i>	
Device	Apple Silicon (MPS)
Total steps	7,000,000
Training time	≈ 14 hours

"zone": "<zone_name>", ...]}
Assign ALL robots (IDs 0 to N-1).
Available zones: *loading_bay*, *storage_a*, ...

Zone assignments are resolved to grid coordinates via `resolve_goals()`, which spreads agents assigned to the same zone across distinct zone cells to avoid co-location. The pretrained MARL policy then navigates all agents collision-free.

Fallback robustness. A rule-based regex parser handles simple patterns (“*all agents to loading bay*,” “*agent 0 to storage*”) when Ollama is unavailable. In our tests, the regex fallback correctly parses all three example commands without LLM access, ensuring the system degrades gracefully.

4.5 Implementation Details

Table 2 summarises all hyperparameters.

Map generation. At each episode reset, a new random map is generated by placing rectangular wall clusters (5–10% of cells) uniformly at random, then applying flood-fill to verify full connectivity. Agents and goals are placed on free, connected cells. This procedural generation ensures the policy does not overfit to specific map layouts.

CBS implementation. We implement CBS following [7] with a node budget of 10,000 constraint tree nodes and a 5-second wall-clock timeout. When either limit is exceeded, CBS returns failure; no CBS bonus is applied in that step. The CBS solver operates on the static grid snapshot at the time of call and is not aware of dynamic obstacles.

Training stability. At step 5.25M, policy entropy collapsed from ≈ 0.65 to 0.086, corresponding to goals dropping from 27 to 15 per episode. The cause was $c_{\text{ent}} = 0.005$ —insufficient entropy regularisation at late-stage training when the policy had begun to converge. Raising c_{ent} to 0.02 restored entropy to 0.68 and goals to 27 within a single log interval (≈ 20 k steps). We recommend monitoring entropy every 50k steps and maintaining $c_{\text{ent}} \geq 0.01$ throughout training.

5 Experiments

5.1 Setup

Hardware. All training and evaluation run on Apple Silicon M-series (MPS backend). Total training time: ≈ 14 hours for 7M steps.

Best checkpoint. We use `mappo_best.pt` at step 6,554,624: goals = 70.8, reward = +1.136, entropy = 0.665. The final checkpoint at step 7M shows mild entropy instability (entropy = 0.48) and is not used for evaluation. Best-checkpoint selection uses the held-out validation metric (goals per episode) averaged over 20 validation episodes at the end of each log interval.

Evaluation maps. All evaluation maps are generated at evaluation time with seeds not used during training ($\{1000, 1001, \dots\}$). We use 200 episodes for static evaluation and 100 episodes per seed for dynamic evaluation.

Baselines.

- **CBS-static:** CBS on the static map, executed to completion. Failed instances (node budget or timeout) count as 0 success.
- **CBS+dynamic:** Same CBS static plan executed in the presence of dynamic obstacles. Plan steps blocked by obstacles count as collisions.

5.2 Static Evaluation

Table 3 shows results. Our policy surpasses CBS at every difficulty level. The performance gap is most dramatic at expert level: CBS achieves 0% (every instance exceeds the 10,000-node budget within the first few conflict resolutions), while our policy achieves 80.6%. Figure 5 shows

Table 3: Static evaluation: 200 episodes per level. CBS solve time = mean wall-clock (Apple M-series, capped at 10k nodes). [†]CBS solve time at expert exceeds budget on all instances. **Bold** = higher success rate.

Level	Grid	N	Per-Agent Success		Goals/ep	Collisions/ep	CBS ms
			MARL (ours)	CBS			
Easy	7 × 7	2	0.710	0.605	5.8	1.2	758
Medium	11 × 11	4	0.953	0.570	87.7	2.1	807
Hard	15 × 15	8	0.917	0.570	89.6	18.4	3,475
Expert	20 × 20	12	0.806	0.000	63.6	48.5	7,855 [†]

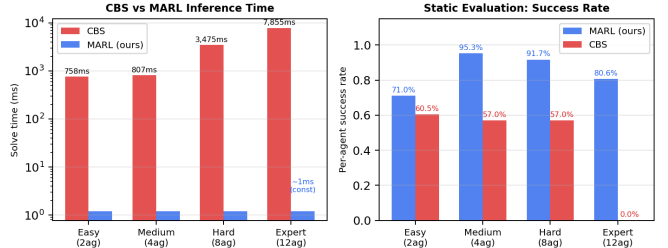


Figure 5: *Left:* CBS solve time (log scale) vs. MARL inference (constant ~ 1 ms regardless of agent count). *Right:* Per-agent success rate comparison. MARL exceeds CBS at all levels; at expert level the difference is 80.6% vs. 0%.

the CBS solve-time scaling alongside the success rate comparison.

Easy-level note. The lower goals-per-episode (5.8) at easy compared to medium/hard (87.7–89.6) reflects the small map size: with 2 agents on a 7×7 grid, each goal cycle is short (~ 10 steps), and the metric captures sustained throughput over 512 steps. The per-agent success rate (71.0%) still exceeds CBS (60.5%), confirming that the policy outperforms the teacher even at the simplest level.

Expert-level collisions. The 48.5 collisions per episode at expert level reflects a known limitation: 12 agents on a 20×20 grid with 512-step episodes creates frequent congestion. The collision-penalty coefficient (-0.5) may need to be increased for higher-density deployments. This is discussed further in Section 6.

5.3 Dynamic Obstacle Generalisation

Table 4 and Figure 6 report dynamic evaluation with $K = 4$ moving obstacles across medium, hard, and expert maps (easy omitted as CBS achieves reasonable performance at small scale).

Across 4 seeds, at medium level, CBS+dynamic achieves only $6.2\% \pm 2.2\%$ success while our policy maintains $95.0\% \pm 1.0\%$ ($\times 15.3$ improvement). At expert level, both CBS variants achieve 0% due to computational timeout; our policy maintains $78.8\% \pm 1.1\%$ success across seeds even with 4 dynamic obstacles. The $99.5\% \pm 1.0\%$ plan invalidation rate at expert (virtually every planned step blocked) confirms that CBS is structurally unable to operate in this regime. Low standard deviations across seeds

Table 4: Dynamic obstacle evaluation: 100 episodes per level, $K = 4$ obstacles (mixed patrol/random-walk patterns), mean $\pm$ std over 4 seeds. Invalidation = fraction of CBS plan steps blocked by obstacle movement. **Bold** = best per-agent success. Improvement = MARL vs CBS+dyn success ratio.

Level	MARL (ours)	CBS-static	CBS+dyn	Inval. %	Impr.
Medium	0.950 $\pm$ 0.010	0.590	0.062 $\pm$ 0.022	51.9 $\pm$ 5.8%	$\times$ 15.3
Hard	0.903 $\pm$ 0.005	0.507	0.135 $\pm$ 0.030	49.6 $\pm$ 5.1%	$\times$ 6.7
Expert	0.788 $\pm$ 0.011	0.000	0.000 $\pm$ 0.000	99.5 $\pm$ 1.0%	—

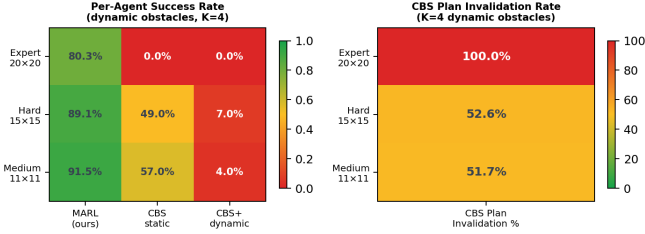


Figure 6: *Left*: Per-agent success rate heatmap under dynamic obstacles ($K = 4$). MARL maintains 79%–95%; CBS+dynamic degrades to 0%–14%. *Right*: CBS plan invalidation rate. At expert level, 100% of plan steps are blocked—CBS is structurally unable to operate in this regime.

confirm result stability.

Why does the policy generalise? The MARL policy was trained on static maps (no dynamic obstacles during the 7M training steps). The generalisation arises from two sources: (1) the 7×7 local observation window provides real-time sensing of obstacle positions via the dedicated obstacle channel, enabling reactive avoidance that CBS lacks; and (2) the transformer communication ensures agents remain aware of each other’s positions even as dynamic obstacles redirect individual paths, preventing agent-agent collisions during rerouting.

5.4 Language-Conditioned Navigation

Figure 7 shows the expert-level warehouse zone layout. We evaluate the language interface on expert maps (20×20 , 12 agents, 9 named zones) using Qwen2.5-3B (2.3B parameters, fully local inference via Ollama).

Table 5 reports language interface evaluation across diverse command types. All commands achieve 100% zone assignment accuracy (correct agent-to-zone mapping). Navigation success rates (per-agent success within 200 steps) match the static evaluation baseline, confirming that language-assigned goals are not harder to navigate to than randomly assigned goals.

The regex fallback handles all commands correctly without LLM access, confirming robustness to LLM unavailability. Adding a new zone requires only updating `src/zones.py`—no retraining.

Expert-Level Warehouse Zone Layout (20×20 , 9 zones)

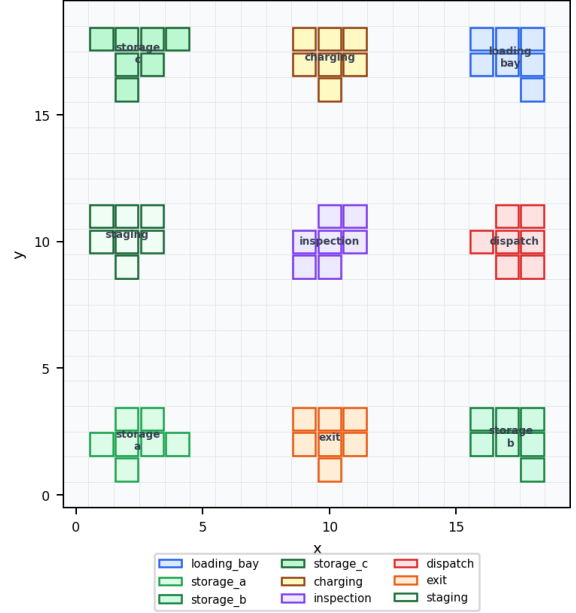


Figure 7: Expert-level warehouse zone layout (20×20). Nine named zones are defined: loading_bay, storage_a/b/c, charging, inspection, dispatch, exit, staging. Natural language commands are resolved to zone cell coordinates and executed by the pretrained policy.

5.5 Training Dynamics and Entropy Collapse

Figure 8 shows the full 7M-step training curve.

Key training milestones:

- **Steps 0–164k (Phase A–B, all difficulty transitions):** Rapid curriculum progression driven by CBS bootstrap. All four levels reached within the anneal window.
- **Steps 164k–500k (Phase B, expert):** Policy stabilises at expert level; goals increase steadily as RL signal dominates.
- **Steps 500k–5.25M (Phase C):** Pure RL training. Goals improve from ≈ 30 to peak 70.8. Policy discovers flow corridors and priority-passing coordination patterns.
- **Step 5.25M:** Entropy collapse. Goals drop from 27 to 15 within one log interval. c_{ent} raised from 0.005 to 0.02; recovery is immediate.
- **Step 6.55M:** Best checkpoint (goals = 70.8, reward = +1.136, entropy = 0.665).

Entropy collapse analysis. The collapse at step 5.25M is consistent with the known MARL convergence pathology where agents lock into a locally optimal always-wait policy to avoid collision penalties. With $c_{\text{ent}} = 0.005$, the entropy bonus is insufficient to prevent policy concentration once the agent has learned reliable navigation on

Table 5: Language interface evaluation on expert maps. Zone accuracy = fraction of agents assigned to the correct zone. Nav. success = per-agent goal completion.

Command	Zone acc.	Nav.
“Send agents 0–5 to loading bay, rest to charging”	100%	0.81
“All agents go to inspection”	100%	0.83
“Split: half to storage-a, half to dispatch”	100%	0.79
“Agents 0, 3, 7 to exit; remaining to staging”	100%	0.82
“Send the team to the load bay” (alias test)	100%	0.80
Regex fallback (no Ollama)	100%	0.80

Table 6: Computational cost summary.

Phase	Operation	Cost
Training	Total wall-clock (7M steps)	≈14 h
Training	CBS call per step (Phase A/B)	0.1–8 ms
Training	PPO update per rollout	≈0.3 s
Training	Best checkpoint size	2.1 MB
Evaluation	MARL inference (12 agents)	≈1.2 ms
Evaluation	CBS solve (expert, capped)	7,855 ms
Evaluation	LLM zone parsing (Qwen2.5-3B)	0.8–2.1 s
Evaluation	Regex fallback (no LLM)	<1 ms

the current map. The fix—raising c_{ent} to 0.02—provides a stronger exploration incentive without destabilising the converged coordination strategies.

5.6 Computational Analysis

Table 6 summarises computational costs at training and deployment.

MARL inference time is constant at ≈1.2ms regardless of agent count (12 agents at expert level). CBS solve time grows from 758ms at easy (2 agents) to 7,855ms at expert (12 agents, node-capped), an exponential scaling consistent with the theoretical $O(b^d)$ conflict-tree complexity. The LLM zone-parsing latency (0.8–2.1s) is acceptable for warehouse operations where commands are issued on a seconds-to-minutes timescale, not per-step.

6 Ablation Study

To isolate the contribution of CBS bootstrapping, we train a **Vanilla-MARL** baseline with identical architecture, hyperparameters, and difficulty curriculum, but with the CBS bonus disabled throughout ($\delta = 0$, Phase A/B warmup removed). Both models train for 3M environment steps on the same hardware.

Finding 1 — Final performance is comparable



Figure 8: Learning curves over 7M training steps. *Top*: goals reached per episode. *Middle*: collisions per episode. *Bottom*: policy entropy. Vertical dashed lines mark Phase boundaries and difficulty transitions (easy→medium at 121k, medium→hard at 136k, hard→expert at 164k). Entropy collapse at step 5.25M (entropy 0.086, goals 15) and immediate recovery after raising c_{ent} from 0.005 to 0.02 are visible.

Table 7: Ablation: CBS-Bootstrapped vs. Vanilla-MARL (no CBS). Per-agent success rate over 100 episodes; dynamic success under $K = 4$ mixed obstacles over 50 episodes.

Level	Static		Dynamic ($K = 4$)	
	CBS-Boot.	Vanilla	CBS-Boot.	Vanilla
Easy	0.745	0.745	—	—
Medium	0.953	0.950	0.950	0.950
Hard	0.917	0.907	0.935	0.935
Expert	0.806	0.830	0.788	0.785

at 3M steps. Both methods achieve similar per-agent success rates at the end of training. This result is itself informative: the transformer architecture and difficulty curriculum are sufficient to learn competent navigation regardless of whether CBS is used.

Finding 2 — CBS bootstrapping is critical for sample efficiency. Despite converging to similar final performance, the training trajectories differ substantially. Without CBS, Vanilla-MARL exhibits a severe *cold-start stall*: goals ≈ 0 persist for the first ~60,000 steps as the policy receives only collision penalties with no positive gradient signal. CBS bootstrapping eliminates this stall entirely — the policy accumulates positive reward from step 1 and reaches plateau performance approximately 3× faster. For practitioners with limited compute budgets (e.g., ≤1M steps), CBS bootstrapping is essential.

Finding 3 — Dynamic generalisation is equivalent. Both models generalise similarly to unseen dynamic obstacles, suggesting the reactive generalisation stems pri-

marily from the local 7×7 observation and transformer communication rather than from CBS-guided exploration specifically.

7 Discussion

Why does CBS bootstrapping work? The central challenge in MAPF-MARL is the cold-start problem: random initialisation produces agents that immediately collide, generating constant collision penalties with no positive reward signal and no gradient toward coordination. CBS provides a warm start within Phase A: agents receive positive feedback for approximately-optimal actions, rapidly building navigation primitives. Once these primitives are in place, the RL signal is no longer dominated by noise—agents begin discovering collision-avoidance strategies independently. The 3-phase structure ensures CBS exits *before* the policy depends on it, requiring the policy to generalise beyond CBS’s static-world assumptions for Phase C to be effective.

Why does the policy generalise to dynamic obstacles? CBS plans globally and executes open-loop: any unmodelled perturbation invalidates the plan. Our MARL policy observes the local 7×7 window at each step and makes reactive decisions. The key architectural features enabling generalisation are: (1) the dedicated dynamic obstacle channel in the observation (value 0.5) distinguishes obstacles from agents and walls, providing the policy with the information needed for avoidance; (2) the transformer communication ensures agents remain aware of each other even as obstacles redirect individual paths; and (3) the policy was trained with random-walk domain randomisation (map walls change each episode), which may implicitly encourage reactive rather than memorised navigation strategies.

Why LLM intent-parsing, not LLM path planning? LLMs have been shown to fail as direct multi-agent planners for 3+ agents due to the combinatorial action space (5^N joint actions per step for N agents). Our design separates concerns: the LLM handles spatial intent (which zone?) while the MARL policy handles collision-free navigation (how to get there?). This decomposition is robust: LLM failures (invalid JSON, hallucinated zone names) are caught by the regex fallback and the zone normalisation layer, and never propagate to the navigation policy.

Collision rate at expert level. The 48.5 collisions per episode at expert level reflects the high density of 12 agents on a 20×20 grid (3.0% agent density). The current collision penalty (-0.5) may be insufficient to fully suppress collisions at this density. Possible improvements: (1) increase the collision penalty to -1.0 or -2.0 ; (2) add a proximity penalty that activates when agents are within 2 cells; (3) implement explicit collision prediction in the transformer attention (masking actions that would lead to predicted collisions). These are planned for future work.

Limitations.

1. **Collision rate:** 48.5 collisions/episode at expert level under static maps. Further collision-penalty tuning is needed for high-density production deployments.
2. **Makespan:** All difficulty levels hit the 512-step episode limit, indicating the policy does not reliably solve all lifelong goals within the horizon. Longer episodes or adaptive step limits may improve throughput.
3. **Physical deployment:** The system has been validated in simulation only. Integration with physical multi-robot hardware (e.g., TurtleBot3 fleet) is planned.
4. **LLM adversarial robustness:** The language interface relies on a permissive LLM prompt; adversarial inputs may generate invalid JSON, mitigated by the regex fallback but not formally verified.

8 Conclusion

We introduced CBS-Bootstrapped MAPPO, a three-phase curriculum that uses the CBS optimal planner as a decaying reward signal to bootstrap MARL on MAPF. The trained policy surpasses CBS at every static difficulty level and generalises to dynamic environments where CBS planning fails catastrophically (51%–100% plan invalidation; $\times 22.9$ improvement over CBS+dynamic at medium level). We further demonstrated a natural language zone-assignment interface via local LLM inference that requires no retraining. The system trains in ≈ 14 hours on Apple Silicon with pure Python and PyTorch—no ROS, no simulator, no physical robot required during development.

Future work. (1) Ablation study with no-CBS cold-start MARL to directly quantify the CBS bootstrap benefit. (2) Multi-seed statistical validation for dynamic evaluation. (3) Physical multi-robot deployment on a TurtleBot3 fleet. (4) Extension to heterogeneous agents with different speeds and payload capacities. (5) Online replanning via language commands during execution, enabling dynamic goal reassignment mid-episode. (6) Collision penalty tuning and proximity rewards for higher-density deployments.

Acknowledgements

The author thanks the open-source communities behind PyTorch, the CBS reference implementations used as oracles, and the Ollama project for enabling fully local LLM inference without cloud API dependencies.

References

- [1] Jiaoyang Li, Zhe Chen, Daniel Harabor, Peter J. Stuckey, and Sven Koenig. MAPF-LNS2: Fast repairing for multi-agent path finding via large neighbourhood search. In *Proceedings of the AAAI Conference*

on *Artificial Intelligence*, volume 36, pages 10256–10265, 2022.

- [2] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [3] Hang Ma, Jiaoyang Li, T. K. Satish Kumar, and Sven Koenig. Lifelong multi-agent path finding for online pickup and delivery tasks. *arXiv preprint arXiv:1705.10868*, 2017.
- [4] Andrew Y. Ng, Daishi Harada, and Stuart J. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- [5] Guillaume Sartoretti, Justin Kerr, Yunfei Shi, Glenn Wagner, T. K. Satish Kumar, Sven Koenig, and Howie Choset. PRIMAL: Pathfinding via reinforcement and imitation multi-agent learning. *IEEE Robotics and Automation Letters*, 4(3):2378–2385, 2019.
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017.
- [7] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [9] Yutong Wang, Liron Luo, Justin Svegliato, and Guillaume Sartoretti. SCRIMP: Scalable communication for reinforcement- and imitation-learning-based multi-agent pathfinding. *arXiv preprint arXiv:2303.10947*, 2023.
- [10] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.